RIPHAH INTERNATIONAL UNIVERSITY

Faculty of Computing

DISK USAGE ANALYZER

Project Documentation

Student Name	Mudansuru Auwalu Garba
SAP ID	66292
Course	Operating System
Instructor	Sir. Yawar Abbas
University	Riphah International University

1. Introduction

In modern computer systems, storage management plays a critical role in maintaining system performance and efficiency. As users continuously download software, save documents, store images and videos, available disk space gradually diminishes. Without proper analysis tools, users often have no clear picture of how their storage is being consumed.

The Disk Usage Analyzer is a Python-based desktop application that solves this problem by providing a comprehensive, visual, and interactive way to analyze disk space usage across local folders, USB drives, and cloud storage services. It is built with a graphical user interface (GUI) using Python's tkinter library and renders charts using Matplotlib.

This project is developed as part of the Operating System course and demonstrates real-world application of core OS concepts such as file systems, directory traversal, storage allocation, and file management.

2. Problem Statement

Many computer users do not have clear visibility into how their disk space is being used. Large files, temporary data, cached browser files, and duplicate content can consume storage silently over time. When disk space becomes critically low:

- System performance degrades noticeably
- Applications may crash or fail to launch
- New software installations fail
- Critical system updates cannot complete
- File saving operations fail unexpectedly

There is a clear need for a simple, efficient, and visually intuitive tool that can scan directories, measure storage consumption, identify large files, and present results in a format that helps users make informed decisions about managing their data.

3. Objectives

The primary objectives of the Disk Usage Analyzer project are:

1. Develop a fully functional Python-based Disk Usage Analyzer with a modern GUI
2. Recursively scan any selected directory and all its subdirectories
3. Calculate and display the total disk space consumed by each folder
4. Count the total number of files and folders scanned

5. Identify and list the largest files in a directory
6. Provide graphical visualizations through Bar Charts and Pie Charts
7. Support scanning of USB and removable storage devices
8. Integrate cloud storage scanning for Google Drive and Dropbox
9. Allow users to export a full scan report as a plain text file
10. Present results in a clean, readable, dark-themed user interface

4. Scope of the Project

The Disk Usage Analyzer covers the following scanning domains:

Scan Domain	Description	Dependencies
Local Folder	Any directory on the local file system	Python stdlib only
USB / Removable	Auto-detected removable drives	Python stdlib only
Dropbox	Full Dropbox account via API token	pip install dropbox
Google Drive	Full Drive via OAuth2	pip install pydrive2

5. System Architecture

5.1 File Structure

The project is organized into a clean two-package structure separating business logic from the graphical interface:

```
DiskUsageAnalyzer/
├── main.py                    # Application entry point
├── requirements.txt           # Python dependencies
├── README.md
│
├── core/                      # Business logic (no GUI code)
│   ├── __init__.py
│   ├── disk_scanner.py        # Local & USB scan engine
│   ├── cloud_scanner.py       # Google Drive & Dropbox scanner
│   └── reporter.py            # .txt report exporter
│
└── gui/                       # All user interface code
    ├── __init__.py
    ├── app.py                 # Main Tkinter window
```

```
    ├── chart_window.py       # Bar & Pie chart popups
    └── cloud_window.py       # Cloud scan dialog
```

5.2 Module Descriptions

main.py

The entry point for the application. Imports and instantiates the DiskAnalyzerApp class from gui/app.py and calls its run() method to start the tkinter main loop.

core/disk_scanner.py

Contains the core scanning engine. Key components include:

- ScanResult dataclass — holds all results from a single scan (total size, file count, top files, folder sizes, tree lines)
- FileEntry dataclass — represents a single file with its path and size
- scan_directory() — recursively walks a directory tree using os.scandir(), accumulating sizes per folder and tracking the largest files
- get_usb_drives() — platform-aware USB detection for Windows, Linux, and macOS
- format_size() — converts raw bytes to a human-readable string (B, KB, MB, GB)

core/cloud_scanner.py

Provides cloud scanning functions that return the same ScanResult format as local scans, ensuring full compatibility with charts and reports:

- scan_google_drive() — authenticates via OAuth2 using pydrive2, recursively lists all files and folders, accumulates sizes per folder
- scan_dropbox() — authenticates with an access token using the dropbox SDK, recursively lists the full account

core/reporter.py

Exports a ScanResult to a formatted plain-text report file. The report includes a summary section, the top largest files list, and the complete directory tree with sizes.

gui/app.py

The main application window. Built with tkinter, it features a dark theme, a button toolbar, a split pane with a tree view on the left and a summary + top files panel on the right. Scans run in background threads to keep the UI responsive.

gui/chart_window.py

Opens a Toplevel window containing a Matplotlib figure embedded inside tkinter. Supports both horizontal bar charts and pie charts of the top folders by size, with the full Matplotlib navigation toolbar.

gui/cloud_window.py

A modal dialog for cloud scanning. Lets the user select Google Drive or Dropbox, enter their credentials (OAuth2 file path or access token), and launches the scan in a background thread. On completion, it passes the ScanResult back to the main app window.

6. Tools and Technologies

Technology / Library	Purpose
Python 3.10+	Primary programming language
tkinter	GUI framework — built into Python standard library
Matplotlib 3.7+	Bar charts and pie charts with TkAgg backend
threading	Background scan threads to keep UI responsive
os / pathlib	File system traversal and path handling
dataclasses	Clean data models for ScanResult and FileEntry
dropbox SDK	Dropbox cloud scanning via REST API
pydrive2	Google Drive scanning via OAuth2
Visual Studio / VS Code	Development IDE
Windows 10/11	Target operating system

7. Features

7.1 Local Folder Scan

The user clicks Local Folder and selects any directory via a file browser dialog. The application recursively scans all files and subfolders, calculating the cumulative size of each folder from the bottom up. Results are displayed in a tree view (left panel) with folder sizes, and a summary panel shows total size, file count, and folder count. The top 15 largest files are listed separately.

7.2 USB / Removable Drive Scan

Clicking Scan USB triggers automatic detection of removable drives. On Windows, it uses the Windows API via ctypes to enumerate drives and identify removable types. On Linux it checks /media and /mnt. On macOS it checks /Volumes. If no USB is found, the user is prompted to select the drive manually. If multiple drives are detected, a selection dialog appears.

7.3 Cloud Scan

The Cloud Scan button opens a modal dialog offering two cloud storage providers:

Dropbox
Authentication: Access token generated from the Dropbox Developer Console
Scans the entire Dropbox account recursively
Displays folder sizes, file counts, and top files just like a local scan
Install: pip install dropbox

Google Drive
Authentication: OAuth2 via client_secrets.json from Google Cloud Console
Credentials are cached locally after first login (no re-authentication needed)
Scans all files and folders in My Drive
Install: pip install pydrive2

7.4 Bar Chart

Opens a Matplotlib window showing the top folders by size as a horizontal bar chart. Each bar is color-coded and labeled with the human-readable size. The full Matplotlib toolbar is available for zooming, panning, and saving the chart image.

7.5 Pie Chart

Opens a Matplotlib window showing storage distribution as a pie chart. Slices represent the largest folders. A legend on the right side shows folder names and their sizes. Slices smaller than 2% are not labeled to avoid clutter.

7.6 Export Report

Generates and saves a comprehensive plain-text report. The report contains a header with scan metadata, a summary section, the top 15 largest files with full paths and sizes, and the complete folder tree with sizes. The user chooses the save location via a file browser dialog.

7.7 Stop Scan

A stop flag is shared between the main thread and the scanning thread. Clicking Stop sets the flag, and the scanner checks it at each file/folder iteration, stopping gracefully without crashing.

8. Methodology

The development followed a structured approach aligned with Operating System concepts:

11. **Step 1:** Requirements Analysis — identified scanning needs, OS API requirements, and UI expectations based on the project brief
12. **Step 2:** Core Logic First — built disk_scanner.py independently of any GUI, making it testable in isolation
13. **Step 3:** GUI Construction — designed the tkinter layout with a dark theme to match the OS-tool aesthetic shown in the project screenshot
14. **Step 4:** Threading — integrated Python's threading module to run scans in the background so the UI remains interactive during long scans
15. **Step 5:** Cloud Extension — added cloud_scanner.py and cloud_window.py as a clean extension without modifying the core scanning logic

16. **Step 6:** Visualization — embedded Matplotlib charts in a separate Toplevel window with the TkAgg backend for native integration
17. **Step 7:** Export — implemented the reporter module to convert any ScanResult to a human-readable text report

9. How to Run the Project

9.1 Prerequisites

- Python 3.10 or higher installed
- pip (Python package manager)
- Windows 10 or 11 (also works on Linux/macOS)

9.2 Installation Steps

Open a terminal or Command Prompt inside the project folder and run the following commands:

```
# Step 1 — Navigate to the project folder
cd C:\Users\User\Desktop\DiskUsageAnalyzer

# Step 2 — Install core dependency
pip install matplotlib

# Step 3 (optional) — Install cloud dependencies
pip install dropbox          # for Dropbox scanning
pip install pydrive2         # for Google Drive scanning

# Step 4 — Run the application
python main.py
```

9.3 Running in Visual Studio

Since the project uses Visual Studio (not VS Code), the recommended approach is:

18. Open Visual Studio
19. Go to View > Terminal to open the built-in terminal
20. Navigate to the project folder using the cd command
21. Run: pip install matplotlib
22. Run: python main.py

10. Troubleshooting

Problem	Solution
'python' is not recognized	Try 'python3 main.py' or add Python to PATH in Environment Variables
No module named 'matplotlib' No module named 'tkinter' No module named 'dropbox'	Run: pip install matplotlib On Linux run: sudo apt install python3-tk Run: pip install dropbox
No module named 'pydrive2'	Run: pip install pydrive2
Cloud scan: invalid token Google Drive: credentials not found	Regenerate the access token from the Dropbox developer console Ensure client_secrets.json is placed next to main.py
Charts show no data USB not detected	Ensure a scan has completed before clicking Bar/Pie Chart Click Scan USB and choose the drive manually when prompted

11. Expected Outcomes

Upon successful completion and execution of the Disk Usage Analyzer:

- Users can scan any local, USB, or cloud directory and instantly see storage usage
- Total disk usage is reported accurately in human-readable units (KB, MB, GB)
- The largest files are identified and ranked by size
- A visual folder tree makes it easy to identify where space is being wasted
- Bar and Pie charts provide graphical insight into storage distribution
- A complete report can be exported for documentation or sharing purposes
- Cloud integration allows users to audit their online storage without leaving the application

12. Operating System Concepts Demonstrated

OS Concept	How It Is Applied in This Project
File Systems	The scanner reads file metadata (name, size, type) directly from the OS file system layer
Directory Structures	Recursive traversal mirrors the tree structure maintained by the OS for organizing files

Storage Allocation	Sizes are calculated using actual allocated bytes reported by the OS via os.scandir()
File Management	The tool helps users identify and manage files consuming excess storage
Process & Threading	Background threads prevent the GUI from freezing during long scans
I/O Operations	File metadata is read via system calls wrapped by Python's os module
Platform APIs	USB detection uses platform-specific APIs (ctypes on Windows, /proc on Linux)

13. Conclusion

The Disk Usage Analyzer successfully demonstrates the practical application of Operating System concepts in a real-world Python project. It provides a useful, production-quality tool that addresses a genuine storage management problem faced by everyday computer users.

The project goes beyond the minimum requirements by adding cloud storage integration for both Dropbox and Google Drive, graphical chart visualizations, multithreaded scanning, and a polished dark-themed GUI. The clean separation between the core scanning logic (core/) and the graphical interface (gui/) makes the codebase maintainable and extensible.

Through this project, students gain practical experience with file system navigation, OS-level storage APIs, GUI programming, multithreading, and cloud API integration — all of which are directly relevant to the study of modern operating systems.

14. References

- Python Software Foundation. Python 3 Documentation — os, pathlib, threading modules. https://docs.python.org
- Matplotlib Development Team. Matplotlib 3.7 Documentation. https://matplotlib.org
- Dropbox Inc. Dropbox Python SDK Documentation. https://www.dropbox.com/developers
- Google LLC. Google Drive API Documentation. https://developers.google.com/drive
- pydrive2 Project. pydrive2 Documentation. https://docs.iterative.ai/PyDrive2
- Silberschatz, A., Galvin, P. B., & Gagne, G. Operating System Concepts, 10th Edition. Wiley.